

Lecture 5 Tree(II), Tree(III)

Tree Traversal

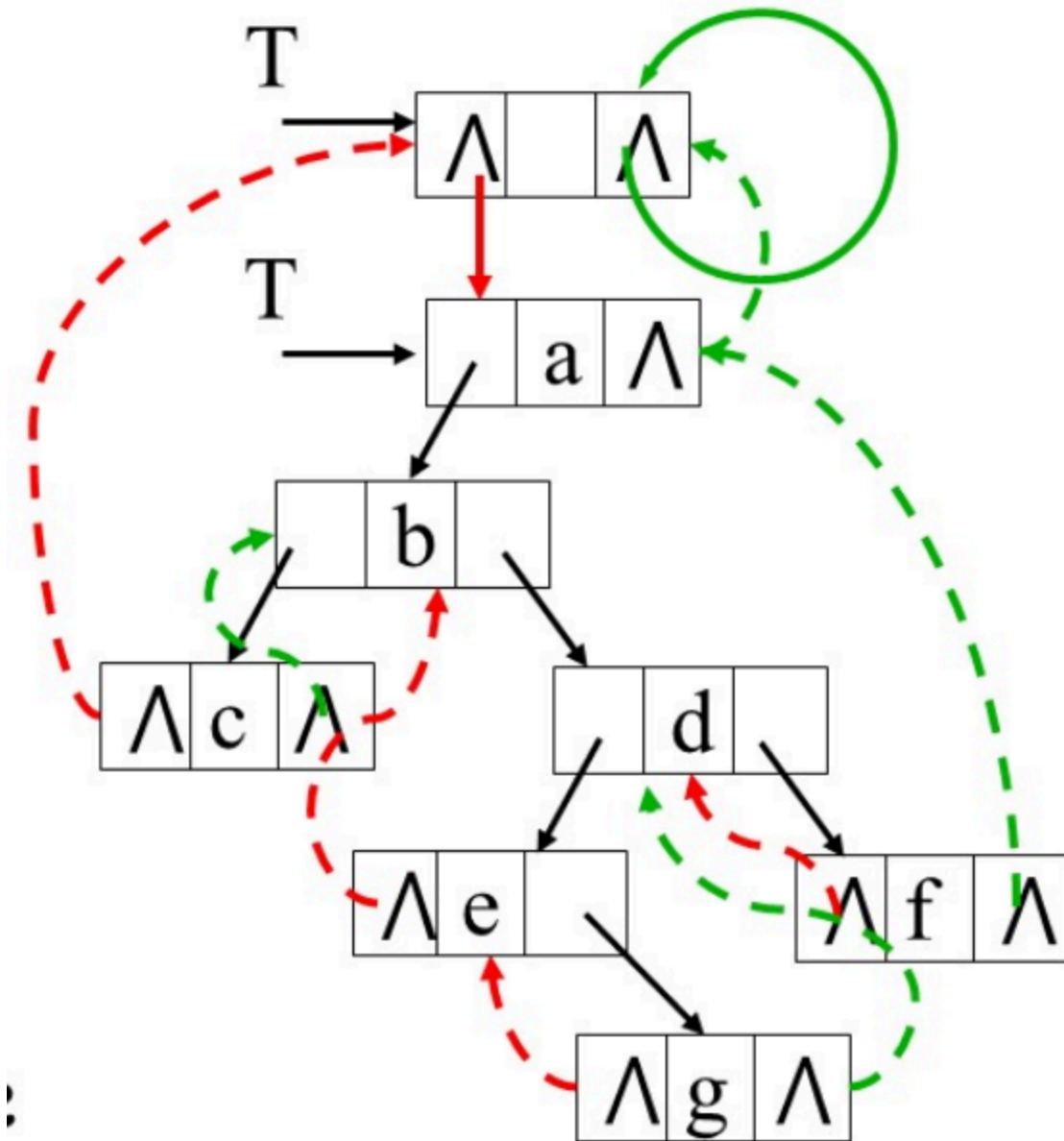
- Preorder
- Inorder
- Postorder

Threaded Binary Trees(Inorder)

Structure

- Rule 1: If the left subtree is null, replace it with a pointer to the inorder predecessor of the node;
- Rule 2: If the right subtree is null, replace it with a pointer to the inorder successor of the node;
- Rule 3: There must not be any loose threads. Therefore a threaded binary tree must have a head node of which the left child points to the first node and the right child points to itself. and we use solid lines represent links and dashed lines represent threads.
To distinguish the link and thread, we need two more variables left tag and right tag to distinguish them.

```
typedef struct threadedBTnode{  
  
    Elementtype data;  
  
    Ttnode *left;  
  
    Ttnode *right;  
  
    int lefttthread;  
  
    int righttthread;  
  
} Ttnode;
```



Solid lines represent links

Dashed lines represent threads

Traverse

Find the first node

The **left most node** in the tree. So we from the head node follow the left link until a node with left thread is true.

Find the next node

- For all the nodes whose **right thread is true**, their **right pointer** points at their successor of the node.
- For all the nodes whose **right thread is false**, the successor is the **left most node in the right subtree** of the node.

Judge the last node

When the iteration reaches **the head node** again.

```
void Inorder_Threaded_BT(TBT t){

    if (!t){

        printf("Error tree.\n");

        return;

    }

    TBT itr = t;

    while (itr->leftthread == 0){

        itr = itr->left;

    }

    while (itr != t){

        cout << itr->data;

        if (itr->rightthread == 1){

            itr = itr->right;

        }else{

            itr = itr->right;

            while (itr->leftthread == 0){

                itr = itr->left;

            }

        }

    }

}
```

```
}  
  
}  
  
}  
  
}
```

Binary Search Tree

Definition

For every node x , all the keys in its left subtree are smaller than key value in x and all the keys in its right subtree are larger than the key value in x .

ADT

Objects: A finite ordered list with elements

Operations:

- Make empty.
- Search element
- Find min
- Find max
- Insert
- Delete
- Retrieve corresponding position

AVL Tree

Height Balanced:

Empty binary tree is height balanced. If T is nonempty binary tree, then T is height balanced if and only if

- T_L and T_R are height balanced
- $|h_L - h_R| \leq 1$.

Balance factor is $h_L - h_R$. In AVL trees, $BF(\text{node}) = -1, 0, 1$.

Insertion in AVL trees

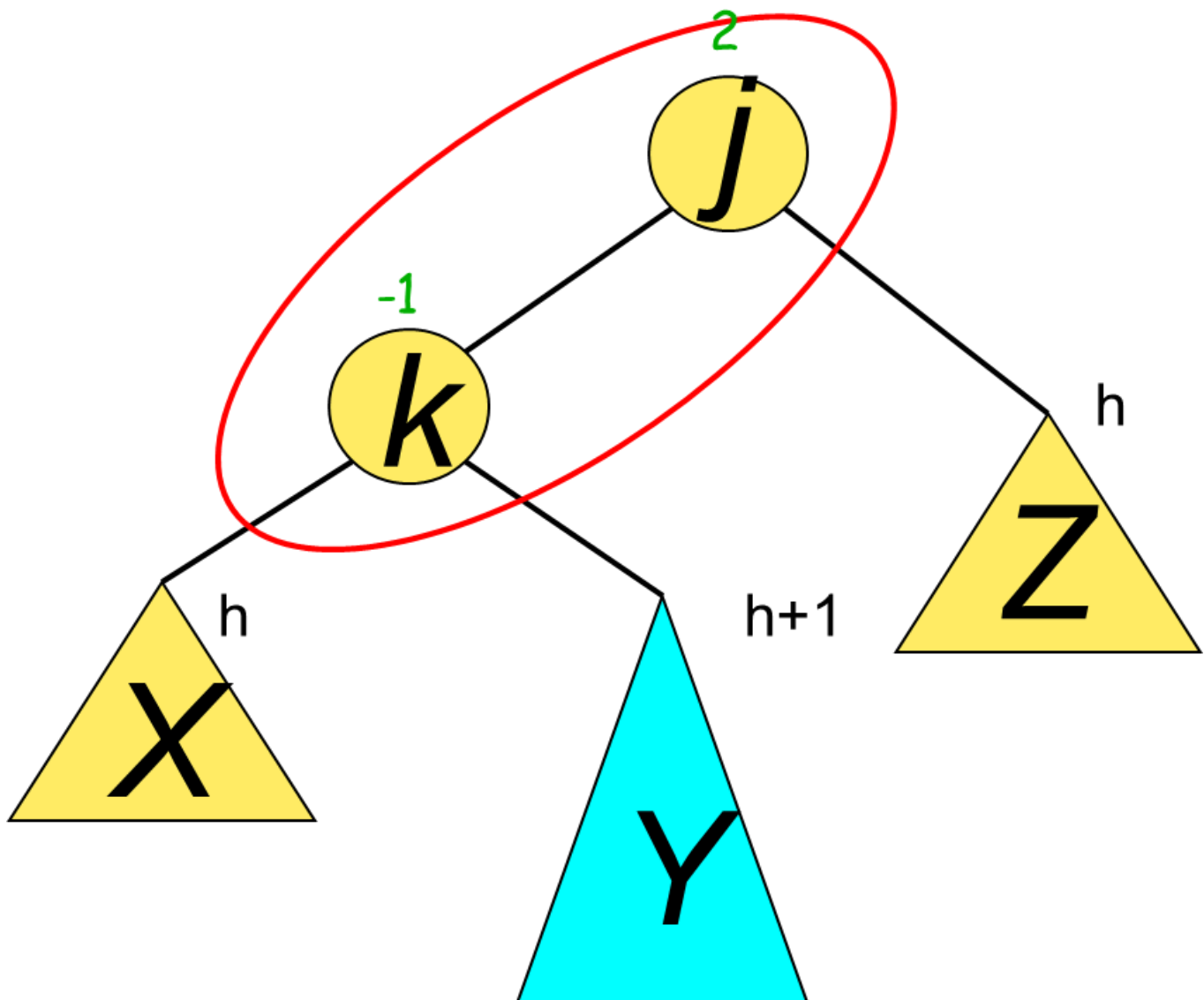
For insert, it may cause unbalance. So after insertion, go back from the inserted node to the root node by node and updating BFs. If unbalanced, rotate to rebalance the tree.

The key is to handle the relationship between trouble finder and trouble maker. Trouble finder is the first unbalanced node from the leaf to the root. Trouble maker is the node that directly causes unbalanced.

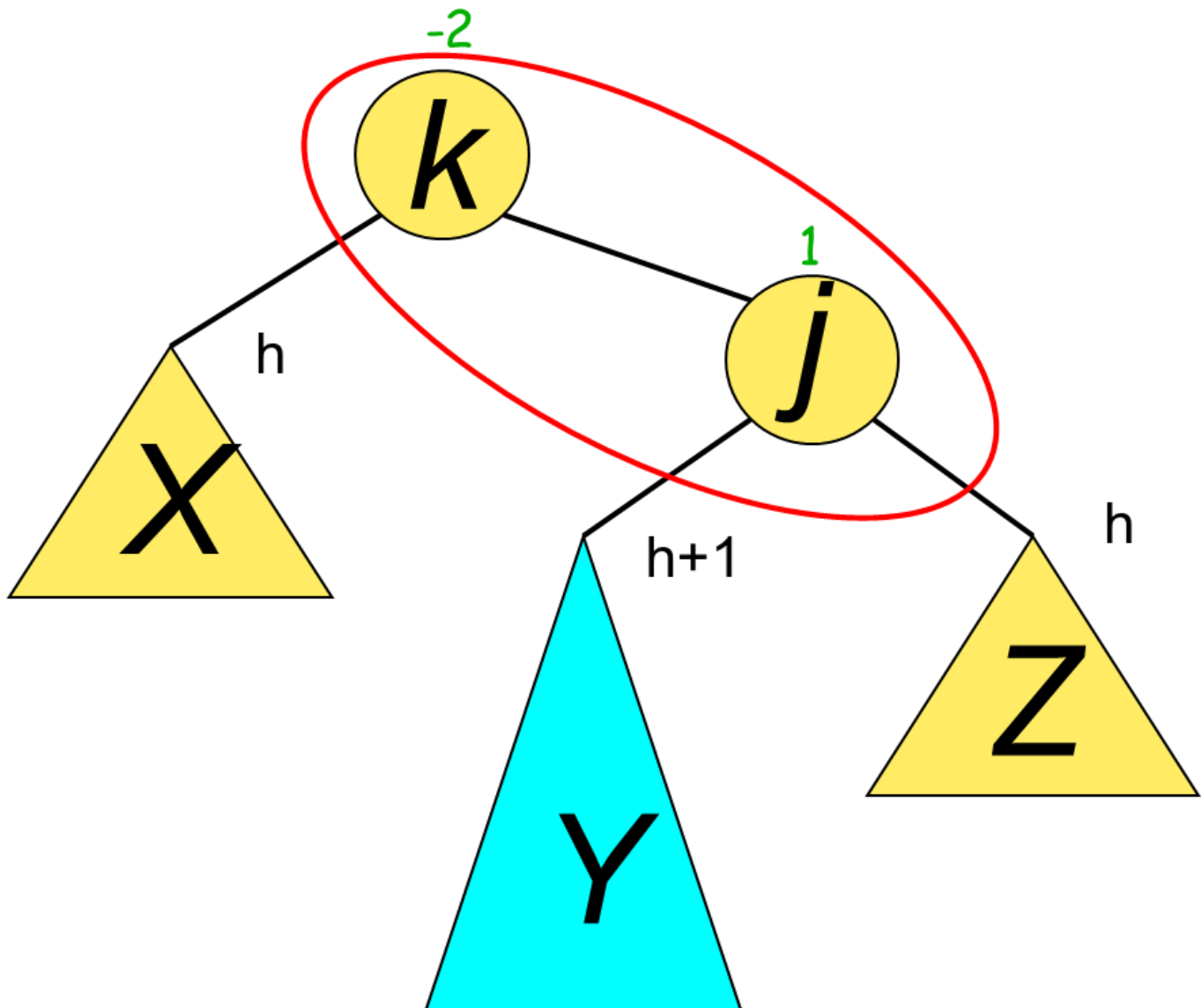
According to the position relationship between trouble finder and trouble maker, there exist four rotations:

- LL rotation(right rotation): trouble maker is in the left child's left subtree of the trouble finder.
- RR rotation(left rotation): trouble maker is in the right child's right subtree of the trouble finder.
- LR rotation: trouble maker is in the left child's right subtree of the trouble finder. (Actually a doubly rotation: RR rotation about k and then LL rotation)
- RL rotation: trouble maker is in the right child's left subtree of the trouble finder.(LL rotation about j and then LL rotation)

LR rotation:



RL rotation:



Deletion

Similar but more complex than insertion:

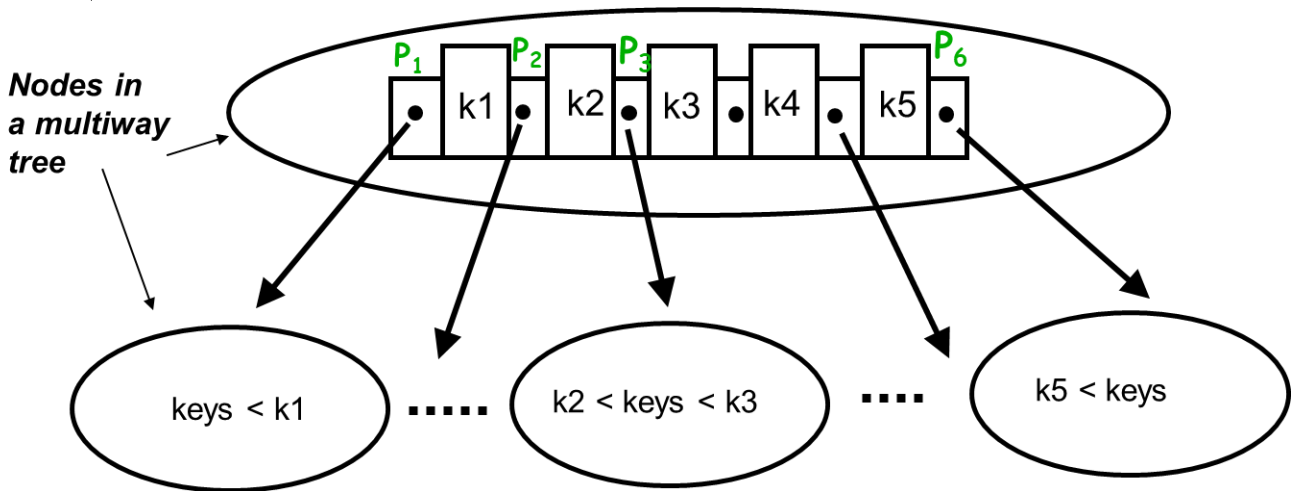
- Rotations and double rotations needed to rebalance
- Imbalance may propagate upward so that many rotations may be needed.

Multi-way Search Trees

Each internal node of a **m -way** search tree:

- has at most m children (links) and $m - 1$ keys
- stores a collections of items of form $P_1, k_1, \dots, P_{m-1}, k_{m-1}, P_m$ where k_i is a key and P_i is a pointer to subtree.

- $k_i < k_{i+1}, 1 \leq i < n$

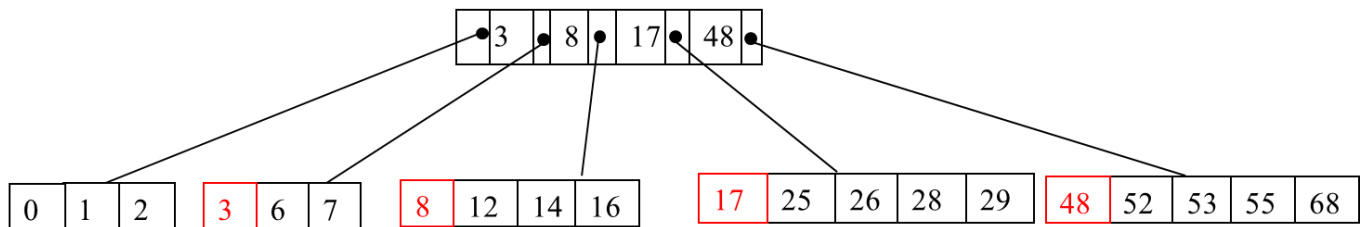


Definition of B-Tree

B-Tree of order m is a m -way search tree such that

- All leaves are on the same level
- Root node has $2 \leq i \leq m$ children, $1 \leq j \leq m - 1$ keys
- Non leaf nodes (except for root) have $\lceil m/2 \rceil \leq i \leq m$ children, $\lceil m/2 \rceil - 1 \leq j \leq m - 1$ keys
- Non root leaf has $\lceil m/2 \rceil \leq j \leq m$ keys.

The height of B-tree is at most $\lceil \log_{\lceil m/2 \rceil} n \rceil$ where n is the number of nodes.



A B-Tree of order 5

Note that the definition in our textbook is a little different:

- Leaves contain **all actual data**
- The **number of keys in leaves** are different to common definition

Searching For a Target in B-trees

Similar to BST.

Insertion

- Find the insertion point **in a leaf** by doing a search.

- If there is room then enter the key
- Else, promote the middle key to the parent and then split the node into nodes around the middle key. ($\lceil k/2 \rceil$ -th)
- If the splitting backs up to the root, then make a new root containing the middle key.

The balance is always maintained.

Deletion

- Find the element in a leaf by searching
- Delete the element and then
 - If it does not cause the node to have too few keys, simply remove the key to be deleted. (remember to update corresponding internal node, omit in the following)
 - If the node contains the minimum number of entries, consider the siblings: if one of these siblings has more than minimum number of entries, borrow one entry from sibling.
 - If all siblings have minimum number of entries, merge the deficient node with one sibling and one key from the parent.
 - The process is propagated upward.